

EX. NO. 5**INTEGRATION WITH EXTERNAL SYSTEMS USING HYPERLEDGER FABRIC****AIM**

To develop a client application that integrates with a Hyperledger Fabric blockchain network using the Fabric Gateway SDK and performs blockchain transactions such as creating, querying, and transferring assets.

SOFTWARE REQUIREMENTS

- Kali Linux / Ubuntu
- Docker
- Docker Compose
- Node.js Version 20 or later
- npm
- Git
- Hyperledger Fabric Samples
- Hyperledger Fabric Binaries
- Fabric Gateway SDK (Node.js)
- Visual Studio Code

PROCEDURE**Step 1: Upgrade Node.js to Version 20+**

Install Node.js version 20 or later using NodeSource. `curl -fsSL`

`https://deb.nodesource.com/setup_20.x | sudo -E bash -`

```
vboxuser@ubuntu:~$ curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
[sudo] password for vboxuser:
2026-08-31 14:44:15 - Installing pre-requisites
Get:1 https://download.docker.com/linux/ubuntu noble InRelease [48.5 kB]
Hit:2 https://deb.nodesource.com/node_20.x nodistro InRelease
Hit:3 http://in.archive.ubuntu.com/ubuntu noble InRelease
Get:4 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:5 http://in.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Hit:6 http://in.archive.ubuntu.com/ubuntu noble-backports InRelease
Get:7 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [976 kB]
Get:8 http://in.archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1,230 kB]
Get:9 http://in.archive.ubuntu.com/ubuntu noble-updates/main Translation-en [287
```

Install Node.js.

```
sudo apt install -y nodejs
```

Check the installed Node.js version. node

```
-v
```

Ensure that the installed version is v20.x.x or later before continuing.

```
vboxuser@Ubuntu:~$ sudo apt install -y nodejs
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
nodejs is already the newest version (20.20.2-1nodesource1).
The following packages were automatically installed and are no longer required:
  bridge-utils containerd docker.io libfwupd2 libgl1-amber-dri libglapi-mesa
  libwoff1 node-busboy node-cjs-module-lexer node-undici node-xtend pigz
  python3-compose python3-docker python3-dockerpty python3-docopt
  python3-dotenv python3-packaging python3-texttable python3-websocket runc
  ubuntu-fan
Use 'sudo apt autoremove' to remove them.
0 upgraded, 0 newly installed, 0 to remove and 43 not upgraded.
vboxuser@Ubuntu:~$ node -v
v20.20.2
```

Step 2: Start the Fabric Network

Navigate to the Hyperledger Fabric test network.

```
cd ~/fabric-dev/fabric-samples/test-network
```

Stop any previously running Fabric network.

```
./network.sh down
```

```
v20.20.2
vboxuser@Ubuntu:~$ cd ~/fabric-dev/fabric-samples/test-network
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ ./network.sh down
Using docker and docker compose
Stopping network
[+] down 10/10
✓ Container ca_orderer Removed 0.5s
✓ Container peer0.org2.example.com Removed 0.5s
✓ Container ca_org1 Removed 0.5s
✓ Container ca_org2 Removed 0.5s
✓ Container orderer.example.com Removed 0.5s
✓ Container peer0.org1.example.com Removed 0.5s
✓ Volume compose_orderer.example.com Removed 0.0s
✓ Volume compose_peer0.org1.example.com Removed 0.1s
✓ Volume compose_peer0.org2.example.com Removed 0.1s
✓ Network fabric_test Removed 0.3s
```

Start the Fabric network and create the application channel with Certificate Authority.

```
./network.sh up createChannel -ca
```

```
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ ./network.sh up create
Channel -ca
Using docker and docker compose
Creating channel 'mychannel'.
If network is not up, starting nodes with CLI timeout of '5' tries and CLI delay
of '3' seconds and using database 'leveldb with crypto from 'Certificate Author
ities'
Bringing up network
LOCAL_VERSION=v2.5.16
DOCKER_IMAGE_VERSION=v2.5.16
CA_LOCAL_VERSION=v1.5.17
CA_DOCKER_IMAGE_VERSION=v1.5.17
Generating certificates using Fabric CA
[+] up 4/4
  ✓ Network fabric_test Created 0.1s
```

Step 3: Deploy the JavaScript Chaincode

Deploy the Asset Transfer Basic JavaScript chaincode.

```
./network.sh deployCC -ccl javascript -ccn basic -ccp ../asset-transfer-
basic/chaincodejavascript
```

```
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ ./network.sh deployCC
-ccl javascript -ccn basic -ccp ../asset-transfer-basic/chaincode-javascript
Using docker and docker compose
deploying chaincode on channel 'mychannel'
executing with the following
- CHANNEL_NAME: mychannel
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
- CC_SEQUENCE: auto
- CC_END_POLICY: NA
- CC_COLL_CONFIG: NA
- CC_INIT_FCN: NA
```

Step 4: Navigate to the Application Directory

Move to the JavaScript application directory.

```
cd ../asset-transfer-basic/application-gateway-javascript
```

List the files in the application directory. ls

Inspect the package.json file to identify the application entry point and available scripts. cat

package.json

Check the main field and the scripts section in the package.json file.

```
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/test-network$ cd ../asset-transfer-b
asic/application-gateway-javascript
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/asset-transfer-basic/application-gat
eway-javascript$ ls
eslint.config.mjs package.json src
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/asset-transfer-basic/application-gat
eway-javascript$ cat package.json
{
  "name": "asset-transfer-basic",
  "version": "1.0.0",
  "description": "Asset Transfer Basic Application implemented in JavaScript u
sing fabric-gateway",
  "engines": {
    "node": ">=20"
  },
  "scripts": {
    "lint": "eslint src",
    "pretest": "npm run lint",
    "start": "node src/app.js"
```

Step 5: Clean Install Node.js Dependencies

Remove the existing node_modules directory and package lock file.

```
rm -rf node_modules package-lock.json
```

Install the required Node.js packages. npm

```
install
```

Step 6: Run the Client Application

Run the application using the available start script. npm

```
start
```

```
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/asset-transfer-basic/application-gat
eway-javascript$ rm -rf node_modules package-lock.json
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/asset-transfer-basic/application-gat
eway-javascript$ npm install
npm warn deprecated eslint@9.39.5: This version is no longer supported. Please s
ee https://eslint.org/version-support for other options.

added 122 packages, and audited 123 packages in 39s

28 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/asset-transfer-basic/application-gat
eway-javascript$ npm start

> asset-transfer-basic@1.0.0 start
> node src/app.js

channelName:      mychannel
```

If a start script is not available, run the application directly using its entry point. node

src/app.js

```
vboxuser@Ubuntu:~/fabric-dev/fabric-samples/asset-transfer-basic/application-gateway-javascript$ node src/app.js
channelName:      mychannel
chaincodeName:    basic
mspId:            Org1MSP
cryptoPath:       /home/vboxuser/fabric-dev/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com
keyDirectoryPath: /home/vboxuser/fabric-dev/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/users/User1@org1.example.com/msp/keystore
certDirectoryPath: /home/vboxuser/fabric-dev/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/users/User1@org1.example.com/msp/signcerts
tlsCertPath:      /home/vboxuser/fabric-dev/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
peerEndpoint:     localhost:7051
peerHostAlias:    peer0.org1.example.com
```

The client application connects to the Hyperledger Fabric network through the Fabric Gateway SDK and performs the required blockchain operations.

Step 7: Initialize and Query the Ledger

The client application invokes the InitLedger transaction to initialize the ledger with sample assets.

It then evaluates the GetAllAssets transaction to retrieve the assets stored in the world state.

The application displays the retrieved asset information.

Step 8: Create a New Asset

The client application submits a CreateAsset transaction to the blockchain network.

The transaction creates a new asset in the ledger with the required asset details.

Step 9: Transfer Asset Ownership

The application submits a TransferAsset transaction to transfer ownership of an existing asset to another owner.

The transaction updates the asset information stored in the world state.

```
*** Result: [
  {
    AppraisedValue: 300,
    Color: 'blue',
    ID: 'asset1',
    Owner: 'Tomoko',
    Size: 5,
    docType: 'asset'
  },
  {
    AppraisedValue: 1300,
    Color: 'yellow',
    ID: 'asset1788187865636',
    Owner: 'Saptha',
    Size: 5
  },
  {
    AppraisedValue: 1300,
    Color: 'yellow',
    ID: 'asset1788187974903',
    Owner: 'Saptha',
    Size: 5
  }
]
```

Step 10: Query the Updated Asset

The client application evaluates the ReadAsset transaction to retrieve the updated asset information.

The returned information is displayed by the application to verify that the ownership change was successfully recorded on the blockchain.

Step 11: Stop the Fabric Network

Navigate back to the Fabric test network directory. `cd`

`../../test-network`

Stop and remove the Fabric network.

`./network.sh down`

APPLICATION WORKFLOW

Start Fabric Network

|



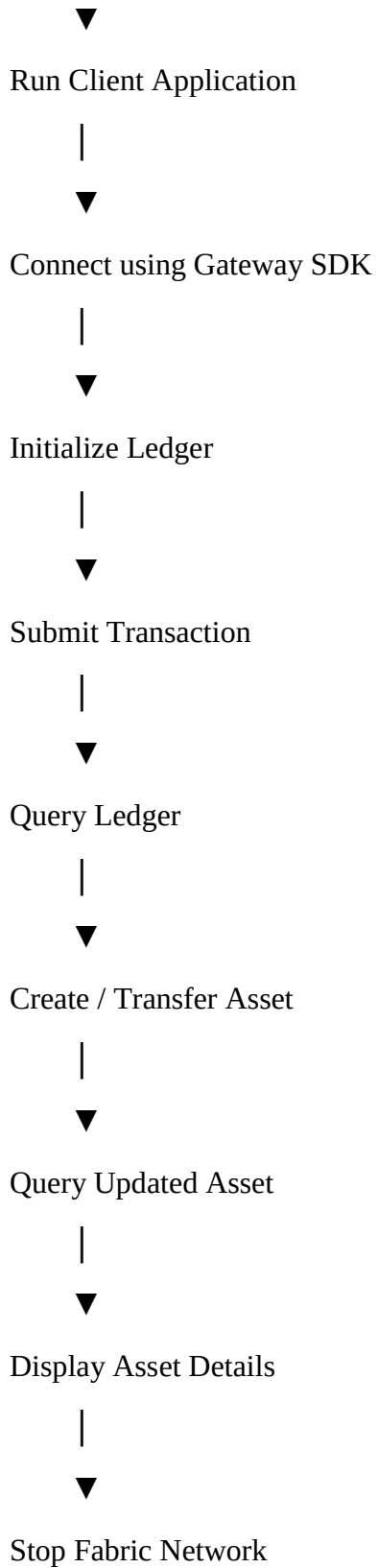
Deploy Chaincode

|



Install Node.js Packages

|



RESULT

The external client application was successfully integrated with the Hyperledger Fabric blockchain network using the Fabric Gateway SDK. The application established a secure

connection, initialized and queried the ledger, created and transferred assets, and retrieved updated asset information, demonstrating how external applications can interact with a permissioned blockchain network.